

# Plotting from first principles: learning to look at numbers

A column of two hundred numbers tells you almost nothing; the same numbers drawn as a curve tell you at a glance. Today we start from what a plot actually is — points placed on two axes — and learn the small set of Matplotlib commands this course uses: lines, scatters, labels, histograms, side-by-side panels and a log axis. No models, no predictions: just the skill of seeing.

Unit 1 · Foundations

80 minutes

Lab 4 · Plot (part 2)

Paper, then laptop

## Where we are

On Friday you learned how NumPy combines arrays of different shapes. You can now compute hundreds of numbers in one line. Today is about the next problem: **looking at them**. Print 200 numbers and nobody can say whether they rise, fall, wobble or jump. Draw them and everybody can.

Matplotlib is Python's standard plotting library. It is very large, and this course uses a small corner of it. Everything in that corner is on this page.

the two imports, at the top of every notebook from now on

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt    # plt is the universal nickname
```

# A plot is a list of points, joined in order

1

Before any code: what does the computer actually do when it "draws a curve"?

Take five values of  $x$  and compute  $y = x^2$  for each. On paper, draw two axes, place the five points, and join them with a ruler.

| x         | 0 | 1 | 2 | 3 | 4  |
|-----------|---|---|---|---|----|
| $y = x^2$ | 0 | 1 | 4 | 9 | 16 |

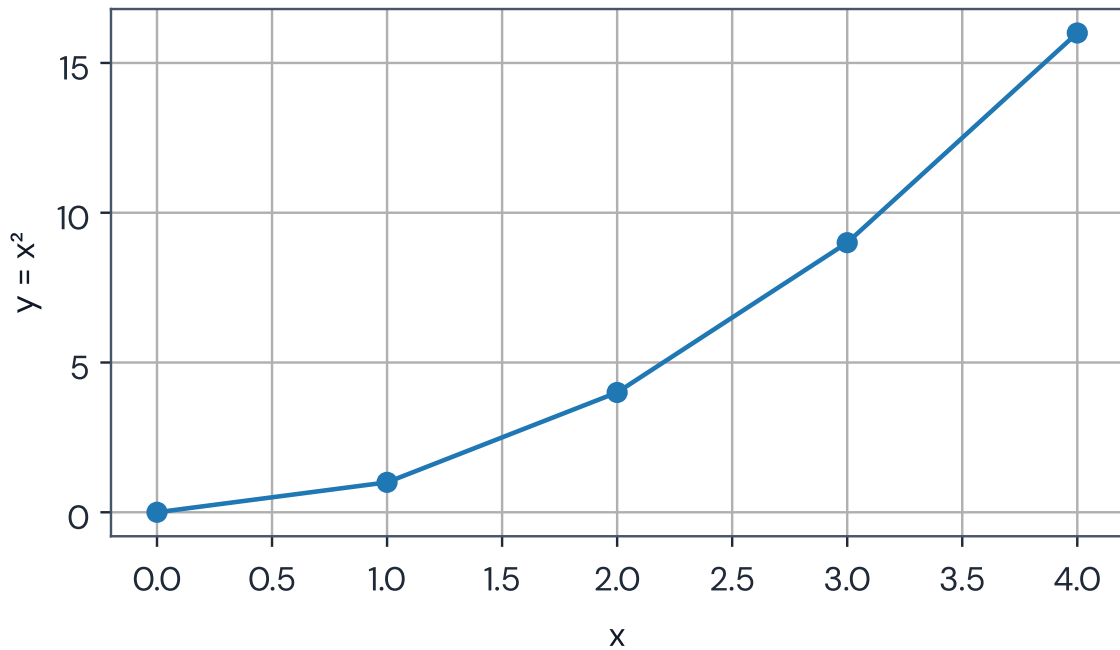
That is exactly what Matplotlib does. Two arrays of the same length go in: the first gives each point's horizontal position, the second its vertical position. Point  $i$  is `(x[i], y[i])`, and consecutive points are joined by straight segments.

your paper drawing, in code

PYTHON

```
x = np.array([0, 1, 2, 3, 4])
y = x ** 2                                     # element-wise: [0 1 4 9 16]

plt.plot(x, y, marker="o")                    # marker="o" draws a dot at each point
plt.xlabel("x")
plt.ylabel("y = x2")
plt.grid(True)
plt.show()
```



Five dots and four straight segments. Look closely: it is not a smooth curve — it has corners at every dot.

#### THREE FACTS THAT EXPLAIN ALMOST EVERY PLOT YOU WILL EVER MAKE

1. `plt.plot(x, y)` draws the points `(x[i], y[i])` and joins neighbours *in the order they appear in the arrays*.
2. Between two points it draws a straight line. A curve looks smooth only because the points are close together.
3. `x` and `y` must have the same length. Matplotlib chooses the axis ranges for you, to fit the data.

fact 3, broken on purpose

ERROR

```
plt.plot(np.arange(5), np.arange(4))
```

**ValueError: x and y must have same first dimension, but have shapes (5,) and (4,)**

The message gives both shapes — just like the broadcasting errors on Friday. Print `.shape` on both arrays, and the fix is usually obvious.

## Resolution and order: facts 1 and 2 at work

Two mistakes everyone makes once. After today, you will know why they happen.

2

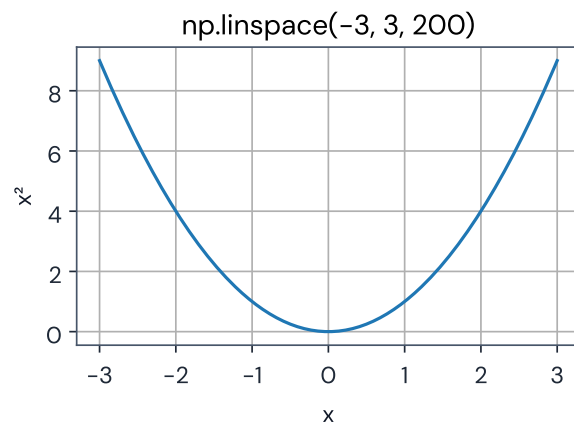
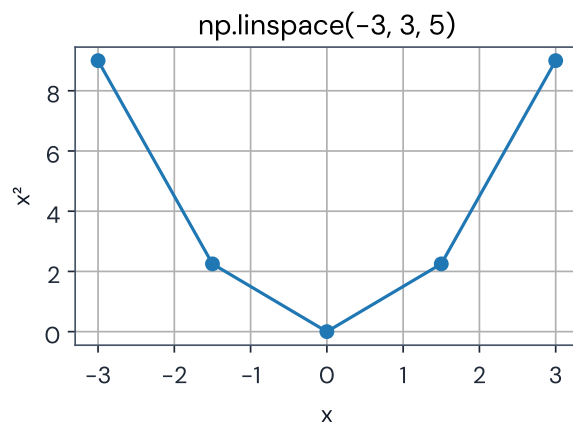
### Resolution: how many points make a smooth curve?

`np.linspace(a, b, n)` gives  $n$  evenly spaced numbers from  $a$  to  $b$ , with both ends included. It is the tool for choosing where to evaluate a function.

`linspace`, then the same function at two resolutions

PYTHON

```
print(np.linspace(-3, 3, 5))  
[-3. -1.5  0.  1.5  3.]  
  
for n in [5, 200]:  
    xs = np.linspace(-3, 3, n)  
    plt.plot(xs, xs ** 2)           # (drawn side by side below; Part 6 shows how)
```



The same function. Five points give a polygon; two hundred give what the eye accepts as a parabola.

How many is enough? Enough that the segments are too short to see. For a function on a laptop screen, a few hundred points is plenty, and NumPy computes them in microseconds — so there is no reason to be stingy.

## Order: what happens when x is not sorted

Real data rarely arrives sorted. Here are twelve x values in random order:

random x values – plot them as a line

PYTHON

```
rng = np.random.default_rng(0)          # seed 0: everyone gets the same numbers
x = rng.uniform(-3, 3, 12)              # 12 random numbers between -3 and 3
y = x ** 2
print(x[:4].round(2))
[ 0.82 -1.38 -2.75 -2.9 ]

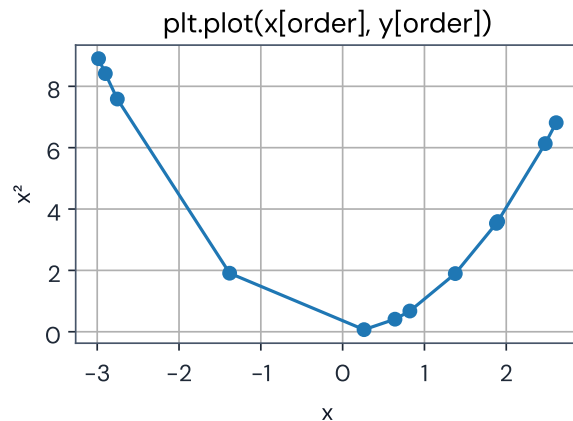
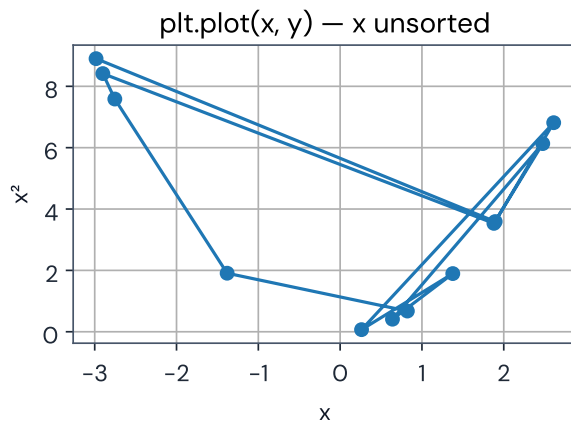
plt.plot(x, y, marker="o")              # left panel below
```

Fact 1 says the points are joined in the order they appear: from 0.82, to -1.38, to -2.75, ... back and forth across the picture. Every point is on the parabola, but the lines between them are a scribble. The fix is to sort — and sort `y` the same way, so each y stays with its own x.

argsort: sort two arrays together

FIXED

```
order = np.argsort(x)                   # the positions of x's values, smallest first
plt.plot(x[order], y[order], marker="o") # right panel below
```



Same twelve points. Left: joined in the order generated. Right: joined after sorting by x.



### New tool: `np.argsort`

`np.sort(x)` returns the sorted values. `np.argsort(x)` returns the *positions* that would sort them — for example, `np.argsort([30, 10, 20])` is `[1, 2, 0]`. Indexing both arrays with the same positions keeps every pair together. Sorting `x` and `y` separately with `np.sort` would break the pairs.

PART 3 · 10 MINUTES

## Labels, titles, legends — and several curves on one plot

3

A plot nobody can read is not finished.

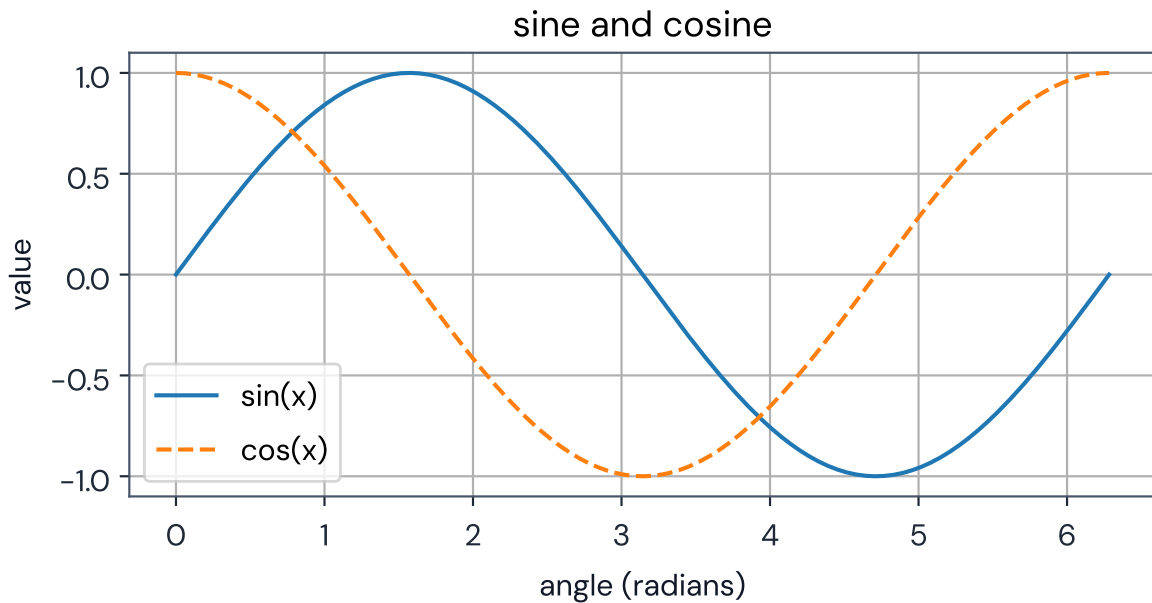
Every `plt.plot` call before `plt.show()` draws on the same axes. Give each one a `label=`, then call `plt.legend()` to show which line is which.

two curves, fully labelled

PYTHON

```
x = np.linspace(0, 2 * np.pi, 200)

plt.plot(x, np.sin(x), label="sin(x)")
plt.plot(x, np.cos(x), label="cos(x)", linestyle="--") # dashed
plt.xlabel("angle (radians)")
plt.ylabel("value")
plt.title("sine and cosine")
plt.legend()
plt.grid(True)
plt.show()
```



Two series, two styles, one legend. The dashed line still reads correctly if the page is printed in black and white.

| Command                                                                 | What it adds             | When to use it                                         |
|-------------------------------------------------------------------------|--------------------------|--------------------------------------------------------|
| <code>plt.xlabel</code> / <code>plt.ylabel</code>                       | names for the axes       | always — include units when there are any              |
| <code>plt.title</code>                                                  | a heading                | when the plot will be seen without the code around it  |
| <code>label=</code> + <code>plt.legend()</code>                         | a key to the series      | whenever there is more than one series                 |
| <code>plt.grid(True)</code>                                             | light reference lines    | when the reader needs to read values off the plot      |
| <code>color=</code> , <code>linestyle=</code> ,<br><code>marker=</code> | appearance of one series | to tell series apart or show where the data points are |



### Label both axes, every time

Two curves with the same shape can mean completely different things depending on what the axes measure. Only the labels tell the reader which situation they are looking at. In this course, treat `xlabel` and `ylabel` as part of the plot, not decoration.

## Scatter: points that should not be joined

When the data is a set of separate observations, joining them invents a path that does not exist.

4

`plt.scatter(x, y)` takes the same two arrays as `plot` but draws only the dots. Because nothing is joined, the order no longer matters.

A small experiment that needs nothing but random numbers: throw 400 random points into a  $2 \times 2$  square, and colour the ones that land inside the circle of radius 1. `inside` is a *boolean mask*: an array of True/False, one per point. Indexing with it, `px[inside]`, keeps only the positions marked True.

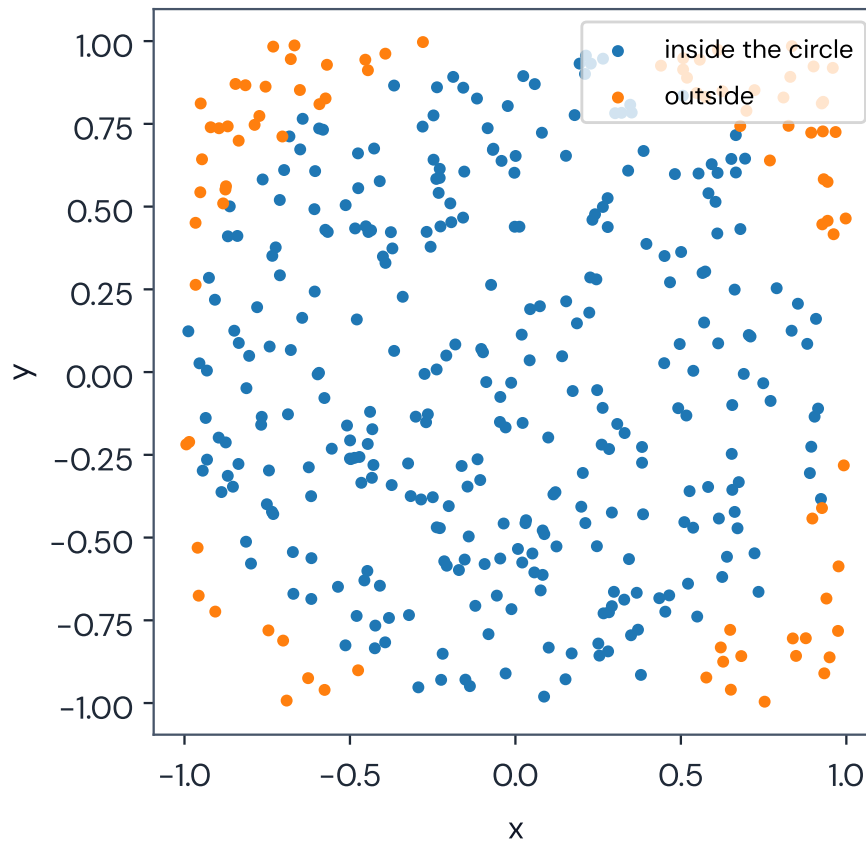
random points, two colours

PYTHON

```
rng = np.random.default_rng(1)
px = rng.uniform(-1, 1, 400)
py = rng.uniform(-1, 1, 400)
inside = px**2 + py**2 ≤ 1           # True where the point is in the circle

plt.scatter(px[inside], py[inside], s=10, label="inside the circle")
plt.scatter(px[~inside], py[~inside], s=10, label="outside")  # ~ means "not"
plt.axis("equal")           # same scale on both axes, so the circle
                             looks round
plt.xlabel("x"); plt.ylabel("y"); plt.legend()
plt.show()

print(inside.sum(), 4 * inside.mean())
312 3.12
```



312 of the 400 points fell inside the circle.

### Why is $4 \times (\text{share inside})$ close to $\pi$ ?

Think about areas: the square, and the circle inside it.

#### ANSWER

The square has side 2, so area 4. The circle has radius 1, so area  $\pi$ . Points are spread evenly, so the share that lands inside is about  $\pi / 4$ . Multiply by 4 and you get roughly  $\pi$ : here 3.12.

Try 400,000 points instead of 400: the estimate gets closer to 3.1416. A picture plus a count turned randomness into a measurement — and the plot is how you check the code did what you think, before trusting the number.



### plot or scatter?

Use `plot` for a quantity that changes smoothly along  $x$  — a function, or a measurement taken step after step. Use `scatter` for separate observations, where joining neighbours would suggest a path that is not there. If in doubt, ask: "does the line between two points mean anything?"

PART 5 · 10 MINUTES

## Counting: histograms and bar charts

Sometimes the question is not "how does  $y$  change with  $x$ ?" but "how often does each value occur?"

5

Roll a die 600 times. Each face should come up about 100 times. And if you add two dice together, some sums are much more common than others — there are six ways to make 7, but only one way to make 2. A picture of the counts makes both facts visible.

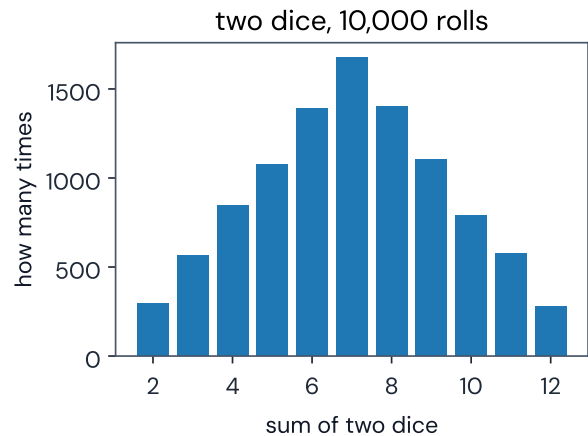
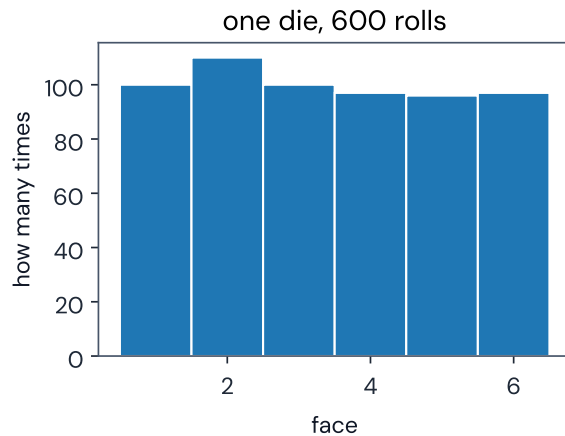
simulated dice

PYTHON

```
rng = np.random.default_rng(2)
die = rng.integers(1, 7, size=600)           # whole numbers 1..6 (7 is
                                             excluded)
two = rng.integers(1, 7, size=10000) + rng.integers(1, 7, size=10000)

# left: let Matplotlib do the counting
plt.hist(die, bins=np.arange(0.5, 7.5, 1), edgecolor="white")

# right: count yourself, then draw bars
counts = np.bincount(two, minlength=13)[2:]  # how many 2s, 3s, ..., 12s
plt.bar(np.arange(2, 13), counts)
```



Left: one die — all faces roughly equal, but never exactly 100 each. Right: two dice — a clear peak at 7.

| Function                                 | Input                       | What it does                                                                         |
|------------------------------------------|-----------------------------|--------------------------------------------------------------------------------------|
| <code>plt.hist(values, bins=...)</code>  | the raw values              | splits the range into bins, counts how many values fall in each, draws a bar per bin |
| <code>plt.bar(positions, heights)</code> | numbers you already counted | draws one bar per position, as tall as you say                                       |

Why `bins=np.arange(0.5, 7.5, 1)`? It makes bin edges at 0.5, 1.5, ..., 6.5, so each whole number sits in the middle of its own bin. With the default bins, Matplotlib chooses ten bins across the range 1 to 6, and some faces would share a bin or straddle an edge. **When values are whole numbers, choose the bin edges yourself.**



### Randomness is visible

The six counts on the left are 100, 110, 100, 97, 96 and 97 — not six 100s. That wobble is normal. Rerun with a different seed and the bars change, but the overall shape does not. Learning to tell "shape" from "wobble" is one of the most useful things a plot teaches.

## Side-by-side panels, a log axis, and saving to a file

6

Three last tools. The first two appear often later in the course.

### Several panels with `plt.subplots`

So far every command went to "the current plot". To draw two plots next to each other, ask for them explicitly. `plt.subplots(1, 2)` returns a *figure* (the whole picture) and two *axes* (the two panels). You then call methods on each panel.

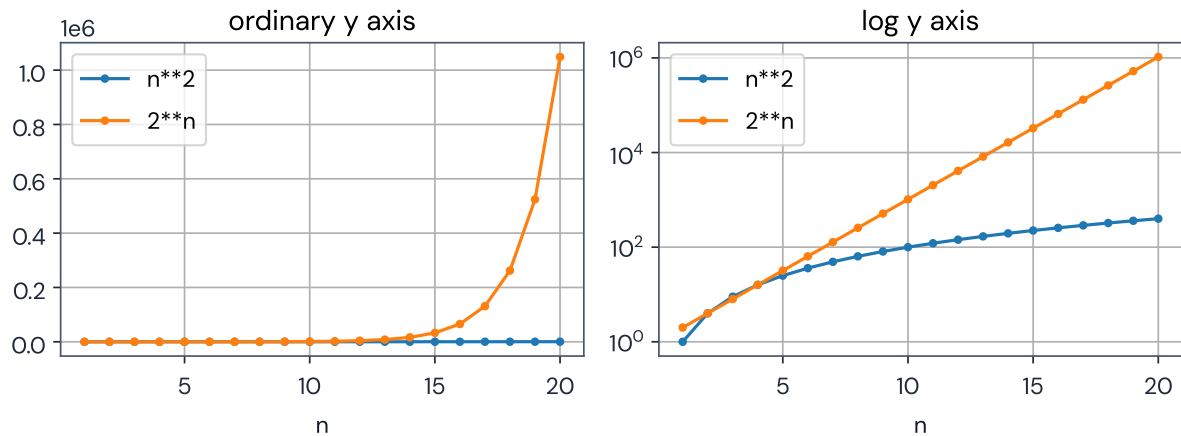
$n^2$  against  $2^n$ , on two kinds of axis

PYTHON

```
n = np.arange(1, 21)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(8, 3.2)) # 1 row, 2 panels

for ax in (ax1, ax2):
    ax.plot(n, n**2, marker=".", label="n**2")
    ax.plot(n, 2.0**n, marker=".", label="2**n")
    ax.set_xlabel("n"); ax.legend(); ax.grid(True)

ax1.set_title("ordinary y axis")
ax2.set_yscale("log") # the only difference between the panels
ax2.set_title("log y axis")
plt.show()
```



Left:  $2^n$  reaches 1,048,576 at  $n = 20$ , so  $n^2$  (at most 400) is squashed flat against the bottom. Right: on a log axis both are readable, and  $2^n$  becomes a straight line.



### The one naming trap

With `plt.` the commands are `plt.xlabel`, `plt.title`. On a panel they gain a `set_` prefix: `ax.set_xlabel`, `ax.set_title`, `ax.set_yscale`. `ax.plot`, `ax.scatter`, `ax.legend` and `ax.grid` keep their names.

## What a log axis means

On an ordinary axis, equal distances mean equal *differences*: 0, 10, 20, 30. On a log axis, equal distances mean equal *ratios*: 1, 10, 100, 1000. Something that doubles at every step — like  $2^n$  — rises by the same distance each step, so it becomes a straight line. Use a log axis when values span many powers of ten, or when you care about ratios ("halved", "ten times smaller") more than differences.

## Saving a figure

save before you show

PYTHON

```
plt.savefig("growth.png", dpi=150)    # a file you can put in a report
plt.show()                          # show() ends the figure – call savefig first
```

If you call `show()` first, the figure is finished and closed, and `savefig` writes an empty picture. `dpi` ("dots per inch") sets the sharpness; 150 is good for documents.

#### THE WHOLE SESSION ON ONE CARD

`plot` joins points in order; `scatter` does not join them; `hist` counts values; `bar` draws counts you already have.

Smoothness comes from many points (`linspace`); a clean line needs sorted x (`argsort`).

Label both axes, add a legend for more than one series, use `subplots` for side-by-side panels and a log axis for values that span powers of ten.

#### LAB 4, PART 2 · 16 MINUTES IN CLASS

## Plot what you computed on Friday

7

Lab 4 is the same handout as on Friday (notebook, HTML and PDF). Today: the plotting part. You turn the timings you measured on Friday into a labelled figure, and make the plots from this page yourself, changing one thing in each.



#### Same rules as before

Finish in the room; put your hand up after three minutes stuck. Every figure you hand in has both axes labelled.

## Before Friday

1. **Finish Lab 4**, both parts.
2. **A plot you will need again.** On one pair of axes, draw `y = x**2` and the straight line `y = 2*x` for `x` from -3 to 3, with a legend. Mark with your eye where the line crosses zero, and where the parabola is lowest. Bring the picture — it returns in Session 10.
3. **Feel the wobble.** Repeat the two-dice bar chart with only 100 rolls, then with 100,000. Write one sentence on what changed and what did not.
4. **Break it on purpose.** Plot the twelve unsorted points from Part 2 with `plt.scatter` instead of `plt.plot`. Explain in one sentence why sorting no longer matters.



### Session 9 — Friday 09 October, 11:30

*The layer is a matrix multiply.* Dot products, matrix multiplication as many dot products at once, and the line  $X @ W + b$  — the  $+ b$  is Session 7's broadcasting — taken apart number by number.